

Министерство науки и высшего образования РФ
Федеральное государственное автономное
образовательное учреждение высшего образования
«СИБИРСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт космических и информационных технологий
институт
Программная инженерия
кафедра

ОТЧЕТ О ПРАКТИЧЕСКОЙ РАБОТЕ №7
Реализация аспектов безопасности

тема

Преподаватель

подпись, дата

Д. В. Грузенкин

инициалы, фамилия

Студент КИ23-17/1Б, 032320072

номер группы, зачетной книжки

подпись, дата

С. В. Зезянов

инициалы, фамилия

Красноярск 2026

СОДЕРЖАНИЕ

1 Задание	3
2 Ход выполнения	4
2.1 Выбор и описание системы для проектирования	4
2.2 Бизнес-требования	5
2.3 Требования пользователей.....	6
2.4 Функциональные требования	8
2.5 Нефункциональные требования	9
2.6 Архитектурный паттерн	9
2.7 Карта контекстов DDD.....	11
2.8 Диаграмма контекста нотации C4.....	12
2.9 Паттерны проектирования	14
2.10 Принципы дизайна и проектирования	17
2.11 Аспекты безопасности.....	19
2.12 Реструктуризация.....	22
3 Выводы.....	29

1 Задание

Выбрать систему, архитектуру которой вы будете разрабатывать в течение семестра. Привести помимо цели и задач работы название системы, краткое описание её назначения и характеристик, а также UML-диаграмму вариантов использования описываемой системы.

Составить список требований для проектируемой системы на следующих уровнях:

- бизнес-требования;
- требования пользователей;
- функциональные требования;
- нефункциональные требования.

Описать источники и методы получения требований.

Выбрать один из приведенных архитектурных паттернов и реализовать в коде без бизнес-логики, реализовать только связи между классами и модулями. Для микросервисной и сервисной архитектур достаточно только структуры проектов и связей между ними, обосновать выбор шаблона в виде ADR (Architecture Decision Record).

Реализовать карту контекстов по методологии DDD, в соответствии с выбранным архитектурным шаблоном. Для смыслового ядра и одной вспомогательной подобласти необходимо привести диаграммы контекста нотации C4.

Реализовать в коде минимум три паттерна проектирования (допустимо в рамках одного класса). Построить UML-диаграмму классов для написанного кода и поместить в отчет. Обосновать выбор реализованных паттернов проектирования и привести 3 примера паттернов, которые в данном случае было бы использовать нецелесообразно, по одному примеру на каждый вид паттернов.

В программном проекте реализовать не менее 3-х принципов SOLID, не менее 4-х принципов организации компонентов (не менее 2-х из каждой группы). Реализованный код отразить на диаграммах кода и компонентов

нотации C4. Принципы SOLID реализовывать до выполнения реализации принципов компонентов.

Необходимо по меньшей мере тремя способами обеспечить повышение безопасности проекта. Описать, почему были выбраны именно эти способы, на какие компромиссы придется пойти, чтобы обеспечить на допустимом уровне выполнение остальных требований проекта. Привести измененную диаграмму компонентов C4, а также диаграмму контейнеров нотации C4, которые отражают внесенные в архитектуру изменения.

Выбрать 2-3 атрибута качества, предоставляющих наибольшую важность в новых реалиях с возросшими требованиями бизнеса (продукт стал популярным и востребованным), и реструктуризовать архитектуру проекта так, чтобы привести эти атрибуты качества к наиболее оптимальным значениям. Изменения внести в код проекта. Описать, к какому архитектурному паттерну был выполнен переход, описать и обосновать причины перехода. Привести для наглядности диаграммы контекста и контейнеров нотации C4 для «старой» и реструктуризированной архитектуры. Обновлённую архитектуру описать в нотации 4+1.

2 Ход выполнения

Заказчиком системы является фитнес-тренер Максим Шмальцев, 30 лет, который самостоятельно ведет занятия и сопровождает клиентов.

2.1 Выбор и описание системы для проектирования

Для разработки архитектуры была выбрана система создания персонализированных тренировочных планов с помощью Telegram и API-сервиса «AI-coach».

Цель системы — помочь людям, заинтересованным в тренировках, фитнесе и здоровом образе жизни.

Система может:

- хранить и использовать для работы данные пользователей;
- генерировать тренировочные планы.

- выдавать рекомендации;
- хранить статистику и историю тренировок;
- мотивировать пользователей.

Для взаимодействия пользователей будет использоваться Telegram, бот в мессенджере.

Для данной системы была разработана диаграмма вариантов использования (use case diagram). Она представлена на рисунке 1.

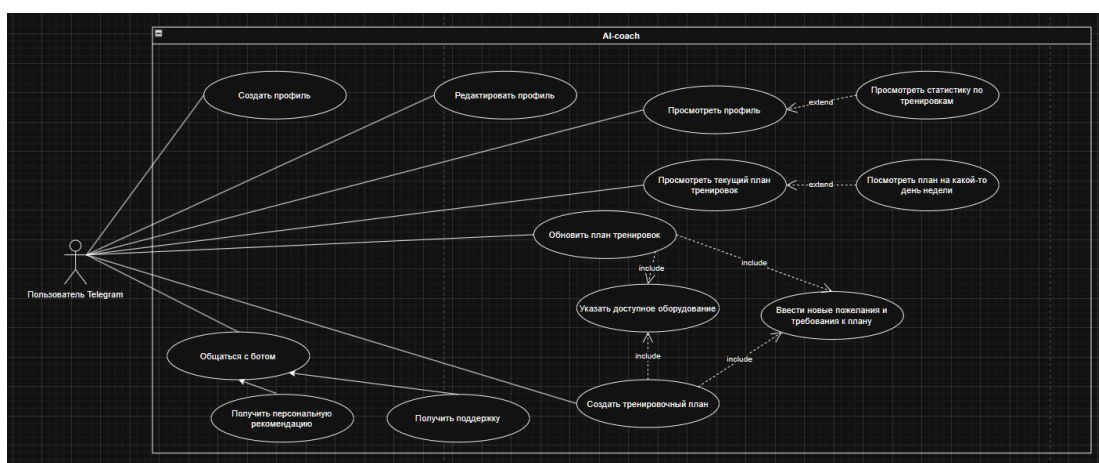


Рисунок 1 – Диаграмма вариантов использования

2.2 Бизнес-требования

Для получения бизнес-требований с заказчиком проведено интервью.

С помощью этого интервью были получены ответы, из которых можно получить бизнес-требования. Интервью можно увидеть в таблице 1.

Таблица 1 – Интервью с заказчиком

Вопрос	Ответ
Расскажите, как вы работаете с клиентами	Я веду персональные тренировки и онлайн клиентов. Обычно клиент заполняет анкету — возраст, вес, цели. После этого я составляю план тренировок и иногда корректирую его
Какие проблемы связаны с этим процессом? Где вы тратите больше времени и сил?	Много задают однотипных вопросов, нужно много времени на составление плана, при этом постоянно держать в голове индивидуальные особенности человека
А как вы составляете планы для тренировок?	Это достаточно трудоемкий процесс, особенно, когда много людей: нужно взять человека, его данные, успехи, личные пожелания и на основе всего этого создать план, который мог бы его удовлетворить

Что бы вы хотели решить с помощью такой умной программы-помощника?	Прежде всего, я хочу сделать так, чтобы клиент сам получал свой план тренировок, чтобы он мог на основе своих показателей получать свои тренировки, чтобы клиент мог получать ответы на свои вопросы автоматически, без моего участия и поддержки. Еще хорошо, когда клиент видит свой рост, это тоже хорошо было бы отразить
Когда вы понимаете, что есть прогресс тренировок, для чего это нужно?	Самый простой и рабочий способ – это самочувствие после тренировок, а также количество этих тренировок. Как таковой системы нет. Нужно это для того, чтобы человек не прекращал заниматься, видел, что у него начинает получаться. Ведь прогресс замечен не всегда замечен сразу
А почему именно Telegram?	Им удобно пользоваться, там обычно мне и пишут клиенты, наверное, только там и пишут, там у меня чаты с ними и свой блог

Выделены следующие бизнес-требования:

- BRQ-1: сократить время работы с клиентом на 40% путем автоматизации составления персонального плана;
- BRQ-2: повысить охват клиентов на 25% благодаря постоянной персональной поддержке и помощи в тренировках через бота;
- BRQ-3: повысить удержание клиентов на 20% с помощью отслеживания прогресса занятий;
- BRQ-4: доступ к системе должен осуществляться через Telegram-бота для наибольшего охвата аудитории.

2.3 Требования пользователей

Среди конечных пользователей системы был проведен опрос.

Опрос состоял из следующих вопросов:

- используете ли вы приложения для фитнеса;
- что представляет трудность для вас в тренировках (несколько вариантов ответа);
- куда вы обращаетесь, чтобы получить совет по тренировкам;
- было бы вам полезно получать персональный план тренировок автоматически;
- кажется ли вам полезным возможность задавать вопросы AI-тренеру;
- хотели бы вы получать мотивационные напоминания для тренировок;

- какая платформа вам была бы удобна для такого фитнес-помощника.

В таблице 2 показаны результаты опроса.

В опросе участвовало 100 пользователей из личного блога тренера.

Таблица 2 – Результаты опроса пользователей

Вопрос	Варианты ответа	Ответы пользователей (в процентах)
Используете ли вы приложения для фитнеса?	1) да 2) нет	1) 60% 2) 40%
Что представляет трудность для вас в тренировках (несколько вариантов ответа)?	1) составить план тренировок 2) сохранять мотивацию 3) следить за прогрессом 4) понять правильную технику 5) не хватает времени	1) 80% 2) 50% 3) 60% 4) 60% 5) 35%
Куда вы чаще обращаетесь, чтобы получить совет по тренировкам?	1) соцсети 2) поиск в интернете 3) тренер 4) знакомые, друзья 5) AI	1) 10% 2) 20% 3) 30% 4) 20% 5) 20%
Было бы вам полезно получать персональный план тренировок автоматически?	1) да 2) нет 3) возможно	1) 60% 2) 10% 3) 30%
Кажется ли вам полезным AI-тренер, который учитывает ваши личные параметры для работы?	1) да 2) нет 3) возможно	1) 70% 2) 10% 3) 20%
Хотели бы вы получать мотивационные напоминания для тренировок?	1) да 2) нет 3) возможно	1) 50% 2) 20% 3) 30%

Выделены следующие пользовательские требования:

- URQ-1: пользователь должен иметь возможность общаться с ботом для получения помощи, советов;
- URQ-2: пользователь должен иметь возможность генерировать для себя план для тренировок;

- URQ-3: пользователь должен иметь возможность создать свой профиль с информацией для тренировок;
- URQ-4: пользователь должен иметь возможность видеть свой личный прогресс в тренировках;
- URQ-5: пользователь должен иметь возможность просматривать свой текущий план тренировок;
- URQ-6: пользователь должен получать мотивационные сообщения;
- URQ-7: пользователь должен иметь возможность указать свои пожелания при создании планов.

2.4 Функциональные требования

С учетом составленных высокоуровневых требований можно составить более низкоуровневые требования – функциональные. Они составляются путем декомпозиции пользовательских требований. Кроме того, можно ориентироваться на общие возможные требования к таким системам.

Выделены следующие функциональные требования, представленные вместе с пользовательскими требованиями, из которых они получены.

URQ-1:

- FRQ-1: общение с ботом через чат для получения советов и рекомендаций.

URQ-2:

- FRQ-2: создание тренировочного плана на неделю на основе данных профиля и дополнительной информации из истории чата с ботом;

URQ-3:

- FRQ-3: создание профиля (рост, вес, возраст, активность);
- FRQ-4: редактирование профиля;
- FRQ-5: удаление профиля;
- FRQ-6: просмотр профиля.

URQ-4:

- FRQ-7: создание отметки о прошедшей тренировке (длительность, самочувствие);
- FRQ-8: просмотр статистики и прогресса;
- FRQ-9: хранение истории тренировок пользователя;

URQ-5:

- FRQ-10: просмотр текущего тренировочного плана на неделю;
- FRQ-11: просмотр тренировки на текущий день.

URQ-6:

- FRQ-12: отправка напоминаний-уведомлений о тренировках пользователю;

URQ-7:

- FRQ-13: генерация планов с использованием дополнительной информацией в сообщении от пользователя;

2.5 Нефункциональные требования

Нефункциональные требования стоят отдельно от других видов требований, они говорят о том, как должна работать система. Они собраны из анализа бизнес-требований, пользовательских и функциональных требований.

Выделены следующие нефункциональные требования:

- NFRQ-1: система должна поддерживать асинхронный режим работы для одновременной работы пользователей без нарушения процесса работы;
- NFRQ-2: пользовательский интерфейс должен быть минималистичен, доступ к основным элементам (профиль, текущий план, создание плана, прогресс, общение с ботом) должен быть доступен в главном окне чата отдельными кнопками или командами.

2.6 Архитектурный паттерн

Ниже представлено обоснование выбранного шаблона в виде ADR (Architecture Decision Record).

Заголовок: Выбор архитектурного паттерна системы

Дата: 21.03.2026

Контекст: разрабатывается чат-бот для фитнес-тренера, который помогает пользователям, мотивируя их, отслеживая прогресс занятий. Он составляет персонализированные тренировочные планы, общается с пользователями, учитывая предпочтения и особенности пользователя, его возможности, желания и потребности, отправляет им уведомления. Можно выделить следующие домены: интерфейс для пользователя через бот, бизнес-логика для работы с планами и пользователями, ML-функционал. Для работы будут использоваться персональные данные и история общения пользователя с ботом, чтобы взаимодействие было максимально персонализировано. Команда небольшая, проект находится на начальном этапе.

Решение: использование архитектуры на основе сервисов (service-based). Всего сервисов будет три: бот, бизнес-логика, связанная с пользователями, тренировками, планами, ИИ-функционал. Сервис бизнес-логики является модульным монолитом.

Напрямую с базой данных может взаимодействовать только сервис бизнес-логики, который управляет данными. Бот будет интерфейсом, с которым работает пользователь, он может обращаться к сервису бизнес-логики. ИИ-сервис будет заниматься только генерацией ответов на основе данных, которые он будет получать. Сервис бизнес-логики обращается к ИИ-сервису. В общем случае, сервис ИИ выступает ключевым сервисом, к которому обращаются остальные.

Последствия: данное решение позволяет разработать систему, в которой сервисы будут фокусироваться на определенном домене (управление данными пользователей, работа с ботом, ИИ-функции). Каждый сервис можно масштабировать независимо и разрабатывать отдельно разными людьми. Дополнительно, это позволяет повторно использовать элементы или перейти к другому мессенджеру, не затронув остальные части системы, или к другому клиенту (мобильное приложение, сайт). Использование слоев упрощает

внесение изменений в отдельном слое внутри каждого сервиса. ИИ-сервис и бот не имеют прямого доступа к базе данных, это обеспечивает целостность данных.

Недостатком являются дополнительные сложности при разработке, так как нужно разработать взаимодействие сервисов. Также взаимодействие между сервисами приводит к определенной задержке в работе, чем если бы модули взаимодействовали напрямую. Потребуется дополнительная работа по созданию инфраструктуры. Тем не менее, этот вариант обеспечивает хороший компромисс между комплексностью и эффективностью. Решение может быть пересмотрено при изменении проекта или требований.

Альтернативы: в качестве альтернативы выступала слоистая монолитная архитектура. Однако она вызывает сильную связанность, отдельные домены смешались друг с другом, что усложнило бы работу. Это не тот вариант, который подходит для приложения с чат-ботом. Другим вариантом была микросервисная архитектура. Микросервисная архитектура сложнее и комплекснее для управления. На данном этапе это не тот вариант, который требуется реализовать. Поэтому компромиссным решением выступает использование сервисной архитектуры.

2.7 Карта контекстов DDD

Относительно домена (персональные фитнес-тренировки) выделены поддомены:

- тренировочные планы (смысловое ядро);
- сопровождение (вспомогательный);
- прогресс (вспомогательный);
- профиль (вспомогательный);
- напоминания (общего назначения).

Можно выделить следующие ограниченные контексты:

- контекст тренировочных планов;
- контекст прогресса;

- контекст напоминаний;
- контекст профиля клиента;
- контекст сопровождения.

На рисунке 2 показана карта контекстов. Связь контекстов выбиралась на основе того, что они расположены внутри одного сервиса. На примере модели «Тренировка» можно понять, что контексты по-разному смотрят на это определение. Для тренировочного плана – это составная единица с набором упражнений и другими параметрами. Для прогресса – это факт о том, что имело или не имело места согласно дате.

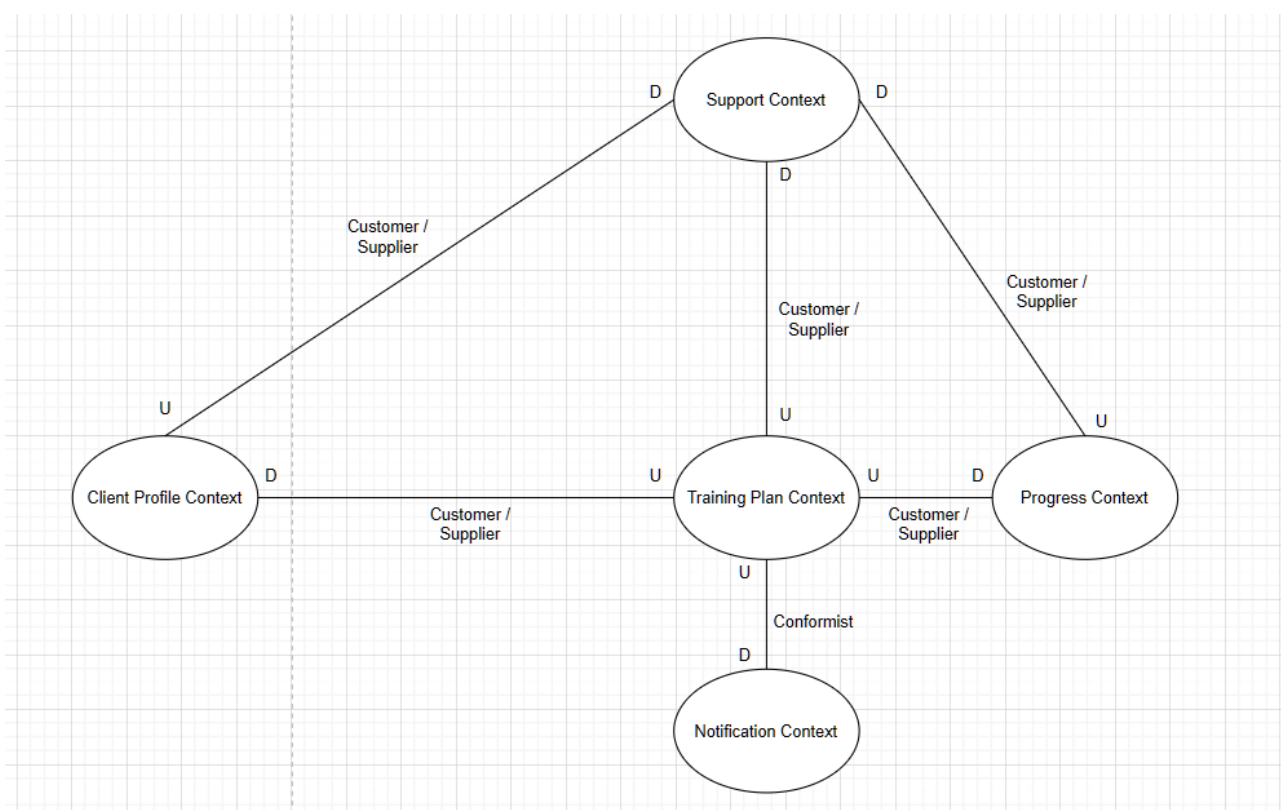


Рисунок 2 – Карта контекстов

2.8 Диаграмма контекста нотации C4

Для смыслового ядра была разработана диаграмма C4 на уровне контекста. Она показана на рисунке 3.

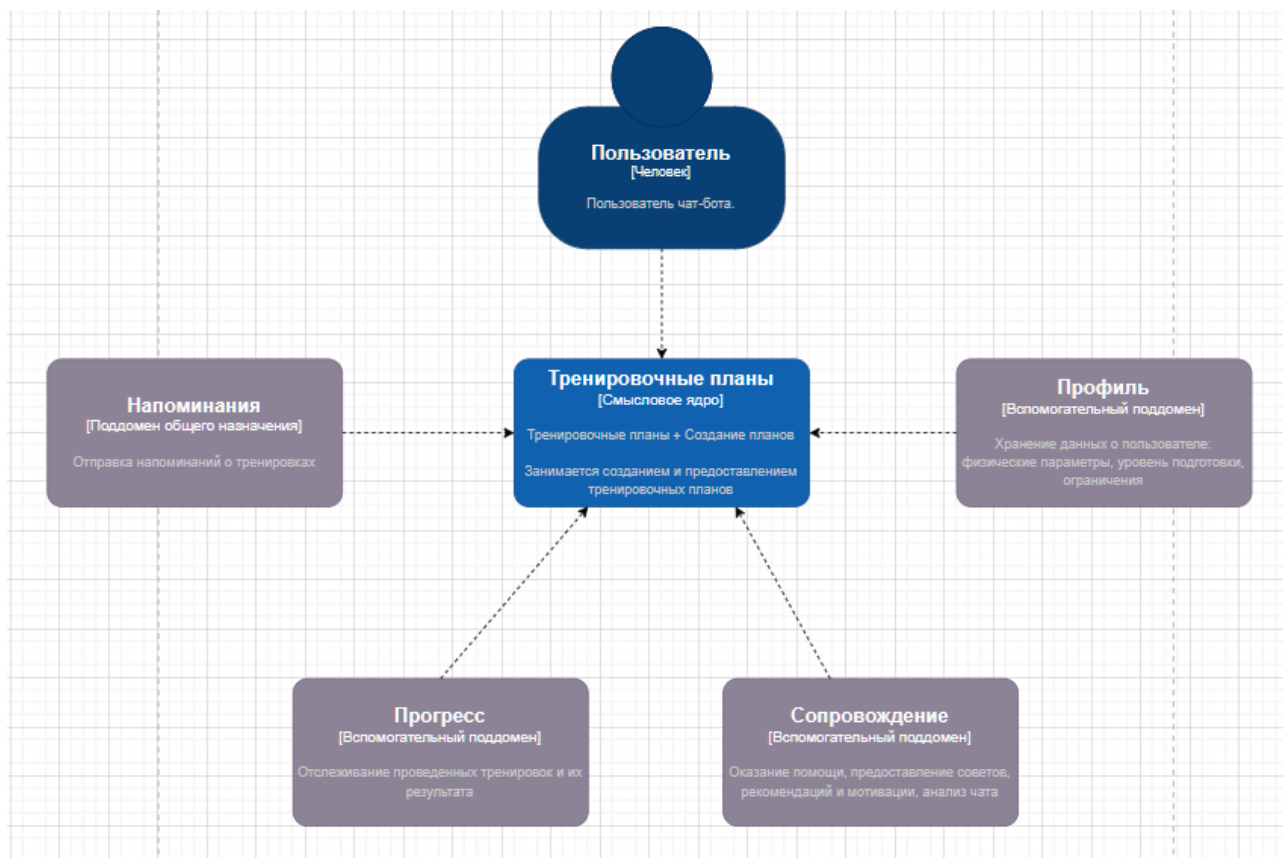


Рисунок 3 – Диаграмма C4 смыслового ядра

Здесь отражено, какие поддомены, системы и люди взаимодействуют со смысловым ядром.

В качестве вспомогательного поддомена для создания диаграммы был выбран поддомен профиля. На рисунке 4 продемонстрирована диаграмма. Заметно, что данный поддомен взаимодействует с меньшим числом доменов системы.

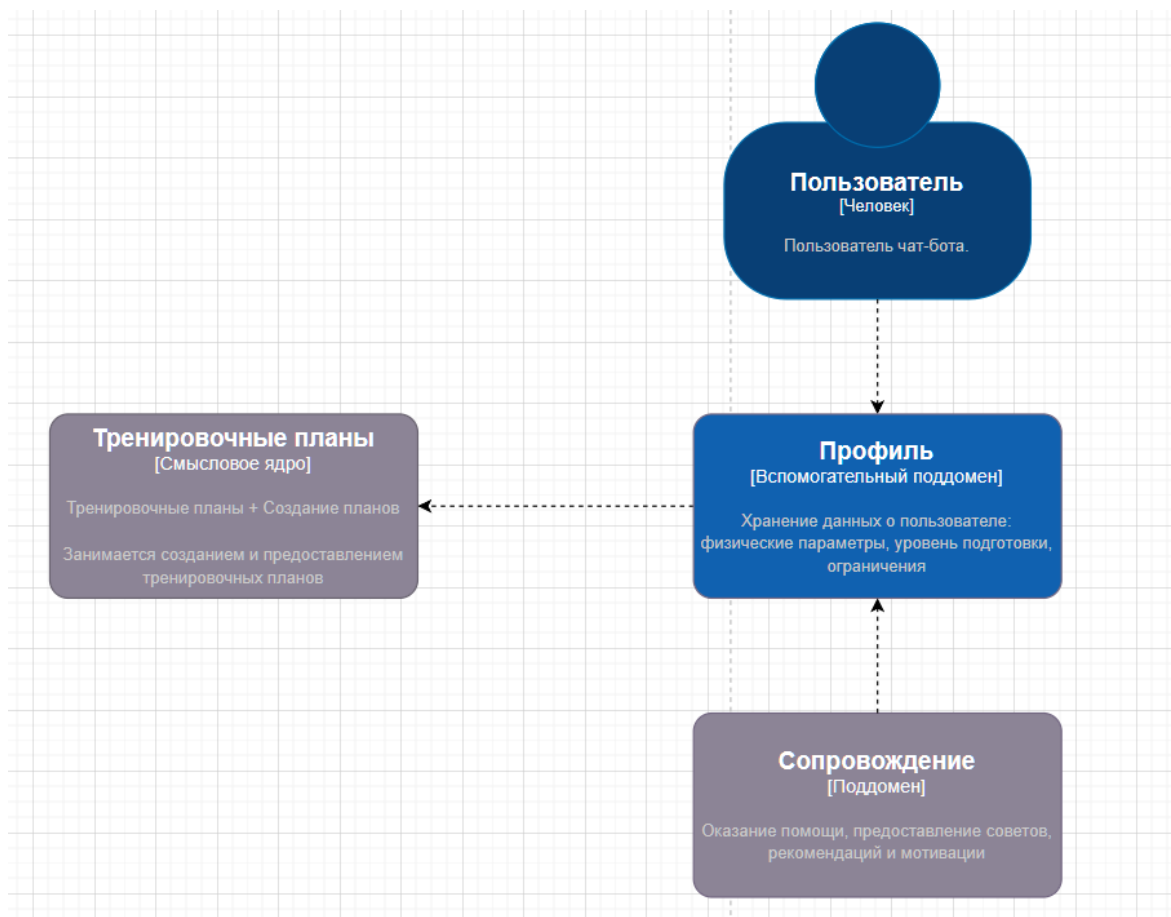


Рисунок 4 – Диаграмма С4 вспомогательного ядра

2.9 Паттерны проектирования

Были использованы следующие паттерны проектирования:

- «Фабричный метод» (порождающий паттерн);
- «Адаптер» (структурный паттерн);
- «Шаблонный метод» (поведенческий паттерн).

Адаптер позволяет организовать общение клиента и сервиса для генерации в установленных форматах: например, с использованием внутренней для модуля доменной модели или формата данных.

Шаблонный метод позволяет изменять алгоритм обращения к сервису генерации, который, по сути, структурно остаются одинаковыми, но отличаются в некоторых шагах. UML-диаграмма классов паттерна показана на рисунке 5.

На рисунке 5 показана UML-диаграмма классов для этих паттернов.

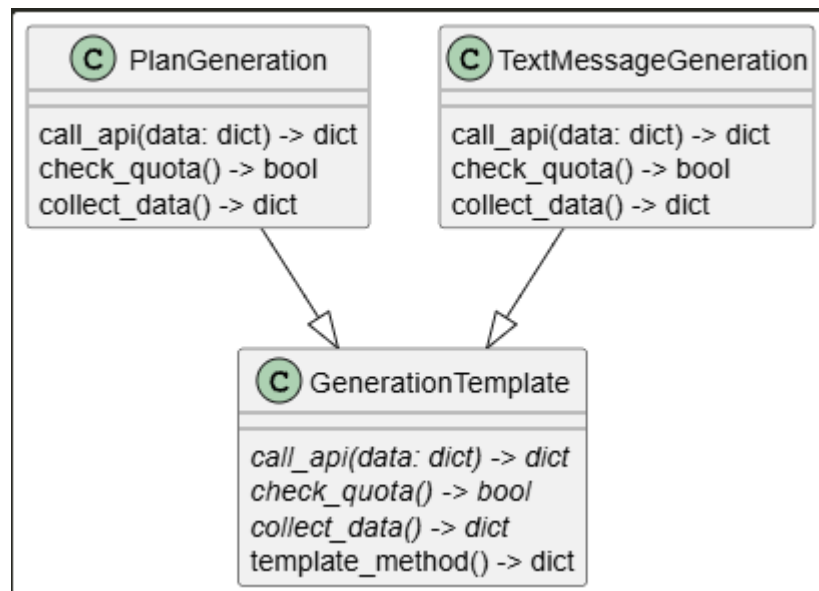


Рисунок 5 – Паттерн «Шаблонный метод»

Фабричный метод применяется для получения ответа от любого источника ИИ (разные модели, допустим, локальная или общедоступная по API). Это позволяет менять источник генерации и добавлять новые способы генерации по мере необходимости в том числе, отделяет логику создания ответа ИИ.

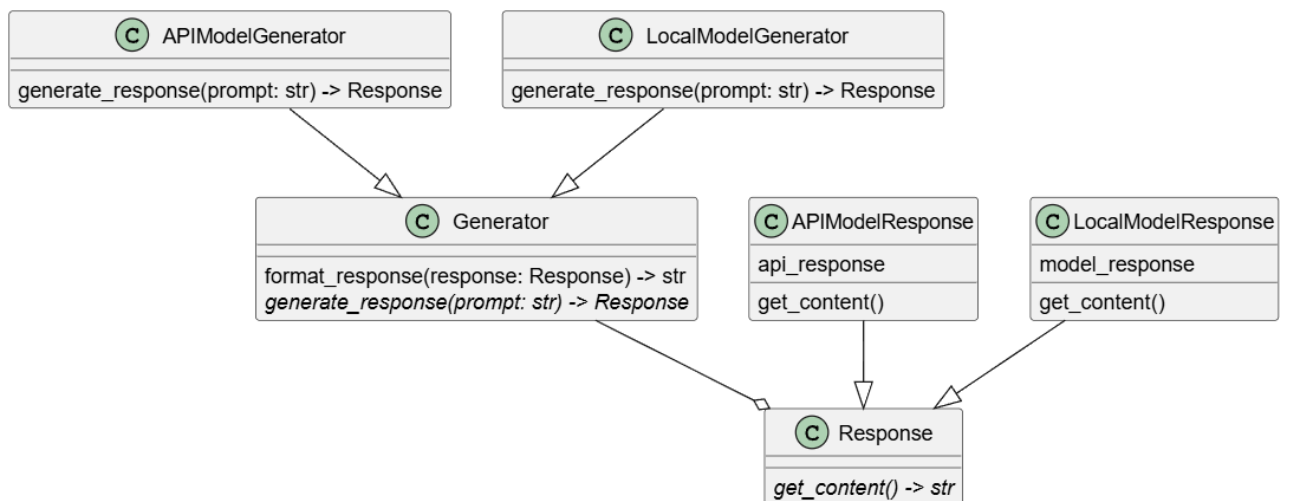


Рисунок 6 – Паттерн «Фабричный метод»

Паттерн «Адаптер» используется для перевода ответа сервиса генерации на язык клиента. Диаграмма представлена на рисунке 7.

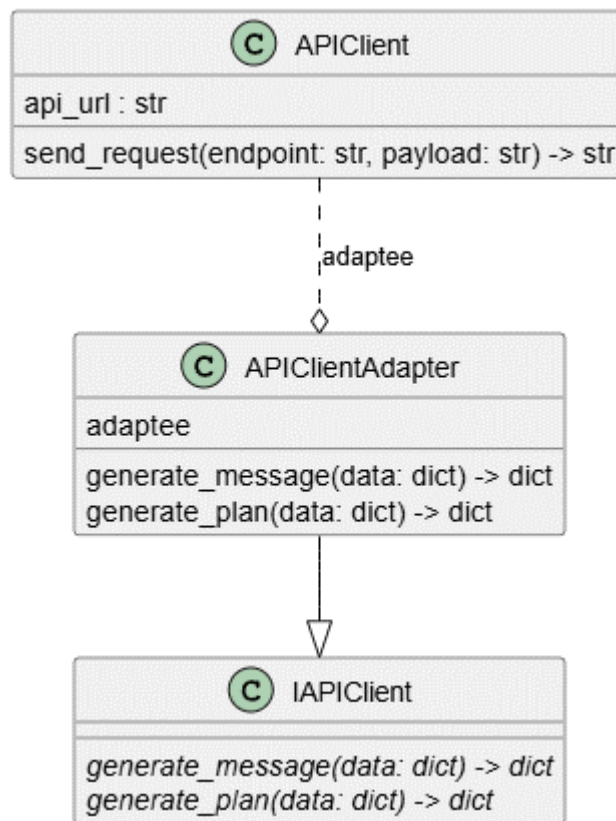


Рисунок 7 – Паттерн «Адаптер»

Рассмотрим паттерны, которые были отвергнуты.

Нецелесообразным в данном случае является паттерн «Итератор». В данной системе не предусмотрены сложные, нетипичные структуры данных или специфические правила обхода. Поэтому данный паттерн не найдет применения в системе.

Паттерн «Фасад» не нашел полноценного применения: система не настолько сложная, поэтому его использование неоправдано.

Паттерн «Легковес» не является подходящим: в системе не предусмотрено большое количество объектов, которые влияют на оперативную память. Также нельзя сказать, что у каких-либо объектов в системе на текущий момент можно выделить общую часть и контекст, чтобы их разделить.

Паттерн «Одиночка» рассматривается, скорее, как антипаттерн некоторыми специалистами. Кроме того, его использование можно заменить внедрением зависимости, что часто реализуется фреймворками, или другими инструментами.

Паттерн «Стратегия» похож на шаблонный метод, тем не менее, использует кардинальное изменение поведения алгоритма, в данном случае актуальнее изменение шагов, когда форма остается той же.

2.10 Принципы дизайна и проектирования

В ходе работы были реализованы следующие принципы SOLID:

- принцип единственной ответственности;
- принцип разделения интерфейсов;
- принцип открытости/закрытости.

На рисунке 8 показана диаграмма кода, в которой отражено использование принципов единой ответственности и принципа разделения интерфейсов. Адаптер для обращения к API был разделен на два класса, каждый из которых реализует конкретный интерфейс.

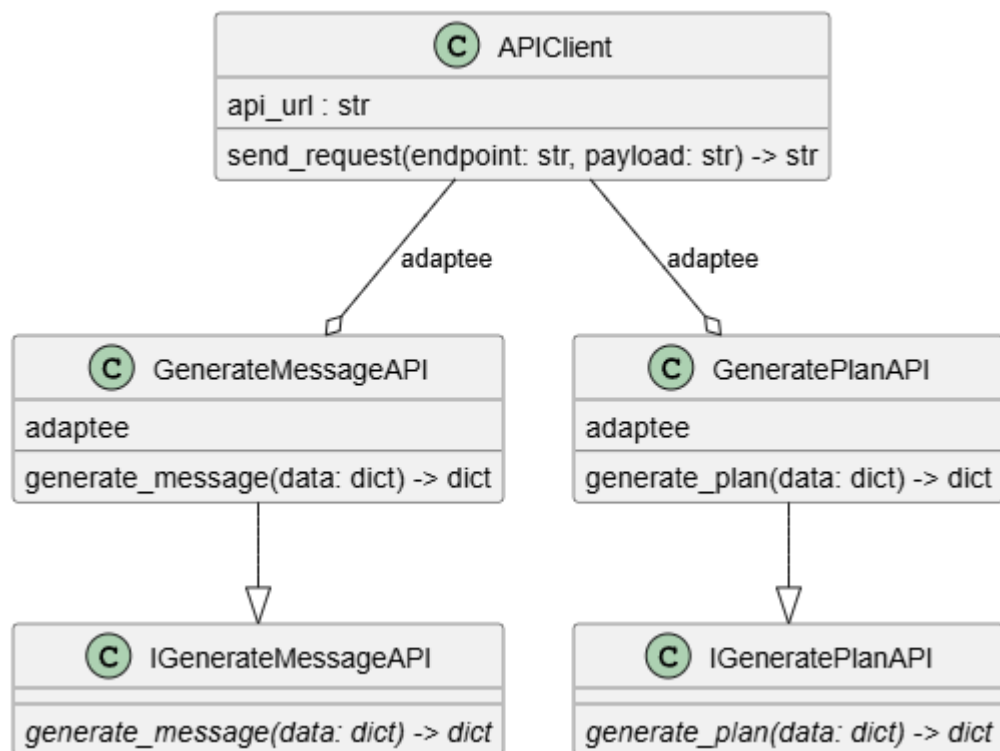


Рисунок 8 – Принципы единственной ответственности и разделения интерфейсов

На рисунке 9 показана диаграмма кода, в которой отражено использование принципа открытости/закрытости. Использование абстракции позволяет закрыть код от изменений, но позволяют расширить его логику.

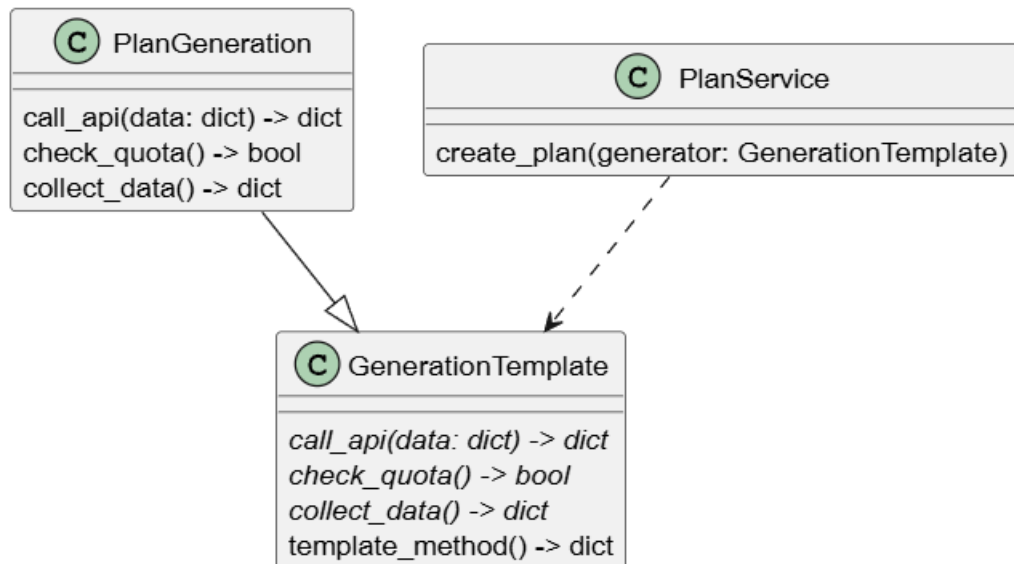


Рисунок 9 – Принцип открытости/закрытости

Установлены следующие компоненты системы:

- users;
- notifications;
- plans;
- progress;
- generation;
- support.

Для организации компонентов были применены следующие принципы:

- принцип согласованного изменения;
- принцип совместного повторного использования;
- принцип ациклических зависимостей;
- принцип устойчивых зависимостей.

Ниже приведены примеры того, как реализуются данные принципы.

Например, компонент «generation» является устойчивым компонентом, от которого зависят другие, менее устойчивые компоненты, сам компонент не

зависит от других компонентов. В то же время в нем используются абстрактные сущности, от которых зависят зависимые от «generation» компоненты.

Компоненты построены так, что в них отсутствует циклическая зависимость.

Компоненты содержат модули и классы, которые работают и изменяются вместе. Так, при изменении логики работы с генерацией потребуется изменить только один компонент «generations».

На рисунке 10 показана диаграмма C4 на уровне компонентов.

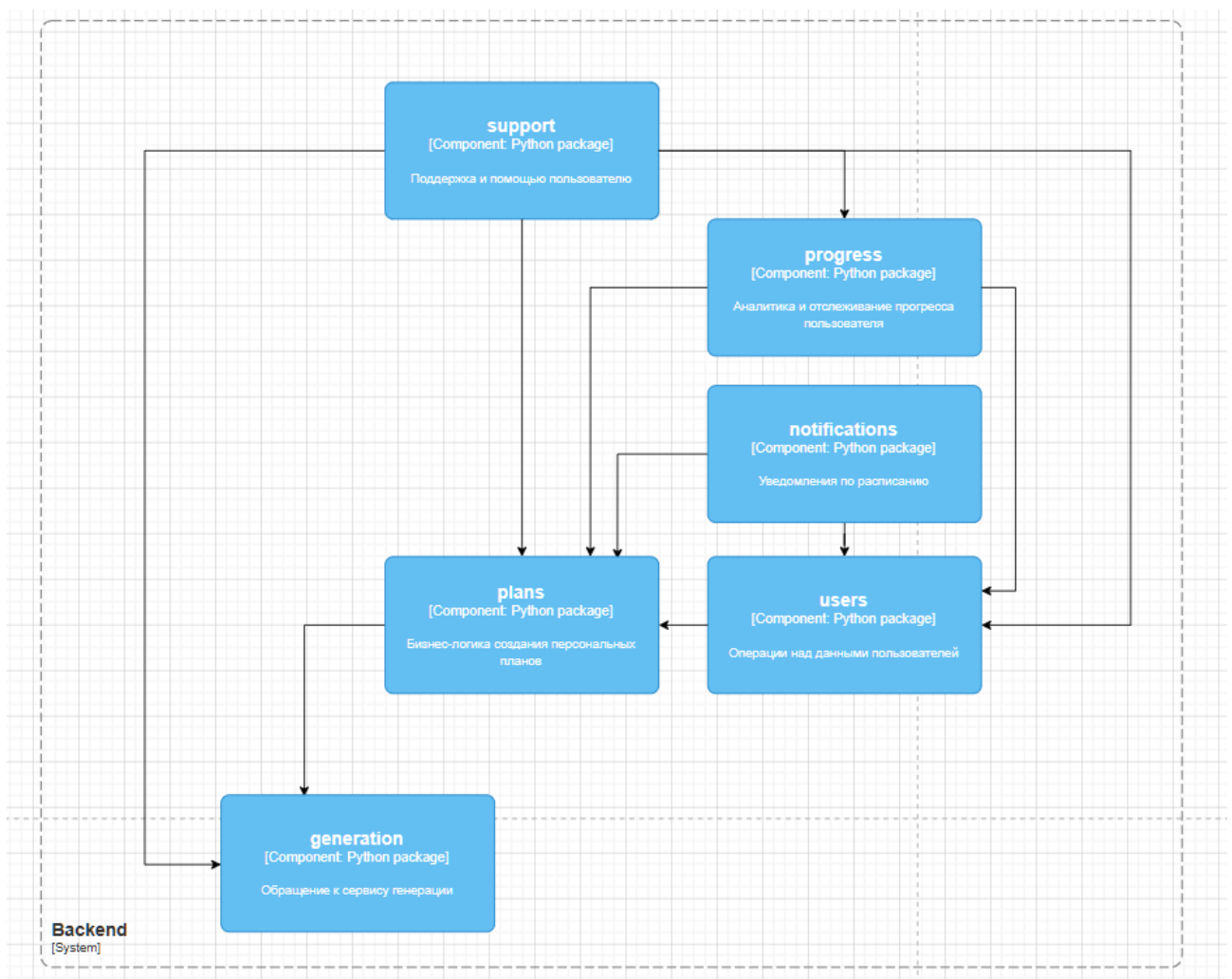


Рисунок 10 – Диаграмма компонентов

2.11 Аспекты безопасности

Были выбраны следующие способы реализации аспектов безопасности:

- модульность;
- симметричное шифрование;

- изоляция компонентов контейнеризацией.

Ниже рассмотрены причины выбора данных способов и возможные компромиссы.

Модульность заложена в архитектуру еще на этапе выбора архитектурного шаблона. Система разбита на модули, у каждого из которых свои границы, четко определенная обязанность и интерфейс для межмодульного взаимодействия. Они работают в своей области. Среди компромиссов возникает дублирование в модулях, сложности с транспортировкой данных (нужны интерфейсы, DTO для передачи данных). Также требуется соблюдать контракт при разработке и не нарушать границы модулей, усложняется взаимодействие, когда задействуются разные модули. Тем не менее, используется одна база данных, что не настолько сильно сказывается на производительности запросов к БД. Также между модулями нет сетевых запросов, что убирает необходимость работать с ними, ускоряет межмодульное взаимодействие

Симметричное шифрование важно для хранения конфиденциальной информации о пользователях, которая будет храниться. Шифрование планируется внутри одного сервиса, поэтому необходимости в обмене ключей нет, так что здесь подходит именно симметричное шифрование: оно быстрее. Вся необходимая логика для работы с шифрованием помещается в отдельный компонент, централизованно занимающийся вопросами защиты данных. Компромисс состоит в уменьшении производительности, связанным с шифрованием. Для решения проблемы следует шифровать только избранные данные, особо чувствительные.

Изоляция компонентов подразумевается с помощью Docker. Так каждый контейнер будет запущен с минимальными привилегиями и ресурсами, в изолированной среде. Компромиссом становится усложнение взаимодействия компонентов системы из-за необходимости настроить сеть. Эта задача становится отдельной ответственностью при разработке.

Обновленная диаграмма компонентов, после реализации способов обеспечения безопасности, представлена на рисунке 11.

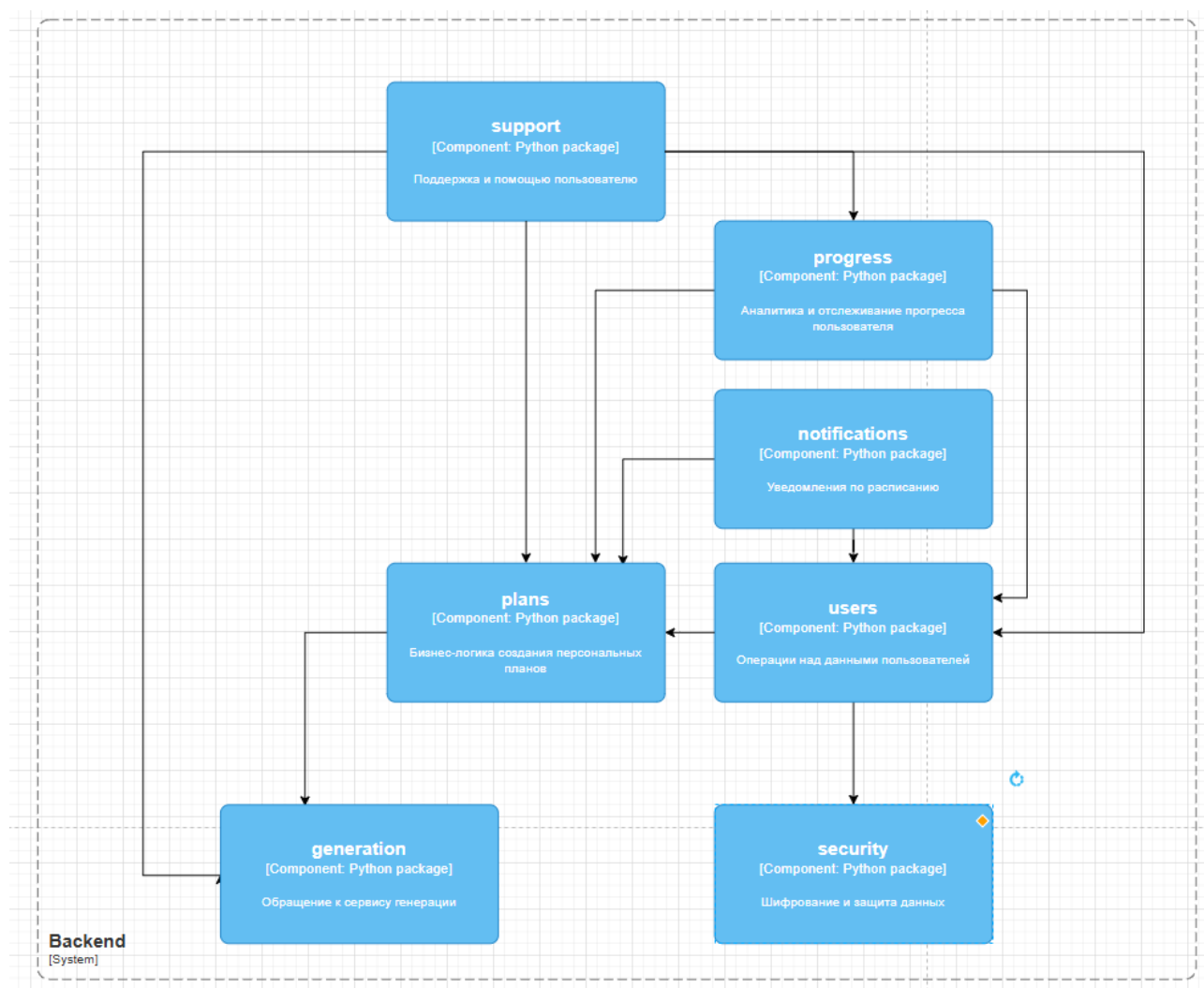


Рисунок 11 – Обновленная диаграмма компонентов

Составленная диаграмма C4 уровня контейнеров представлена на рисунке 12. Описание контейнеров представлено на диаграмме.

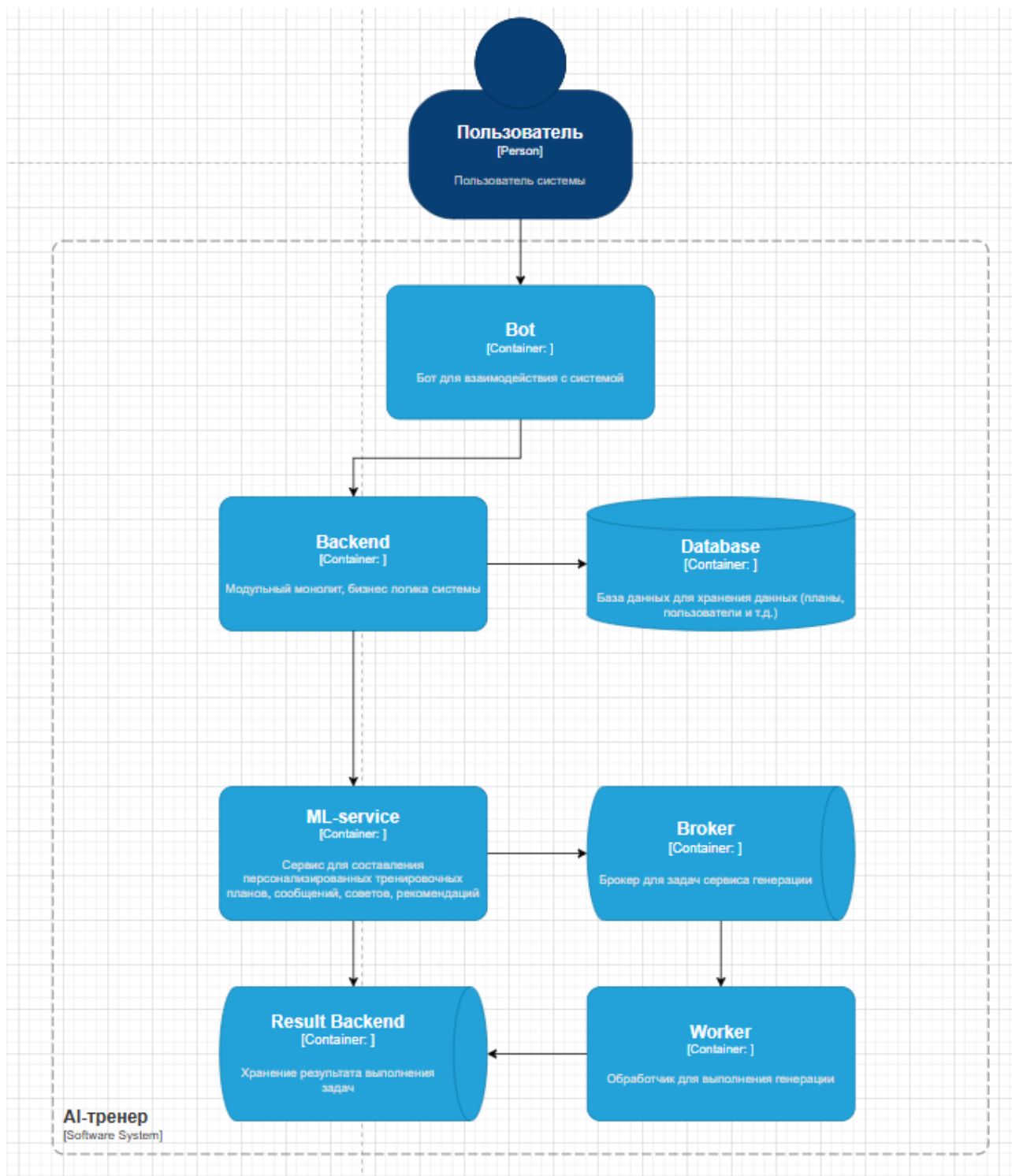


Рисунок 12 – Диаграмма контейнеров

2.12 Реструктуризация

Произошла гипотетическая ситуация: система стала популярно, стало больше пользователей или клиентов у тренера. В связи с этим требуется адаптировать существующую систему под эту ситуацию.

Выбраны следующие атрибуты качества, которые становятся наиболее важными в новых условиях бизнеса:

- надежность – система должна сохранять работоспособность 99.9% времени при высоких нагрузках;
- масштабируемость – система должна быть открыта к увеличению нагруженных подсистем при перегрузке узла.

В связи с этим было принято решение произвести переход к полноценной сервисной архитектуре: сервис бизнес-логики из модульного монолита превращается в сервисную архитектуру, общение между сервисами будет происходить с помощью брокера через события и сообщения.

Это разумный переход, поскольку модульный монолит подготовлен архитектурно к тому, чтобы его можно было разделить на отдельные макросервисы (модули становятся сервисами). Тогда мы получаем масштабируемость за счет возможности горизонтального масштабирования в дополнение к вертикальному, что также повышает надежность: под растущей нагрузкой система будет выполнять свои функции. Нужные сервисы можно дублировать, если они испытывают чрезмерную нагрузку. Балансируется нагрузка входящего трафика между серверами, не происходит перегрузка отдельного узла. Повышается устойчивость за счет того, что выход из строя одного компонента не прерывает работу всей системы.

Переход к другому виду монолитной архитектуры не рассматривается, так как она не решает проблему масштаба. Подход с единой БД создает проблему, когда к одной БД множество запросов от нескольких сервисов.

В связи с этим происходят некоторые изменения в диаграмме контейнеров нотации C4.

Диаграмма контекста остается прежней.

На рисунке 13 представлена обновленная диаграмма контейнеров.

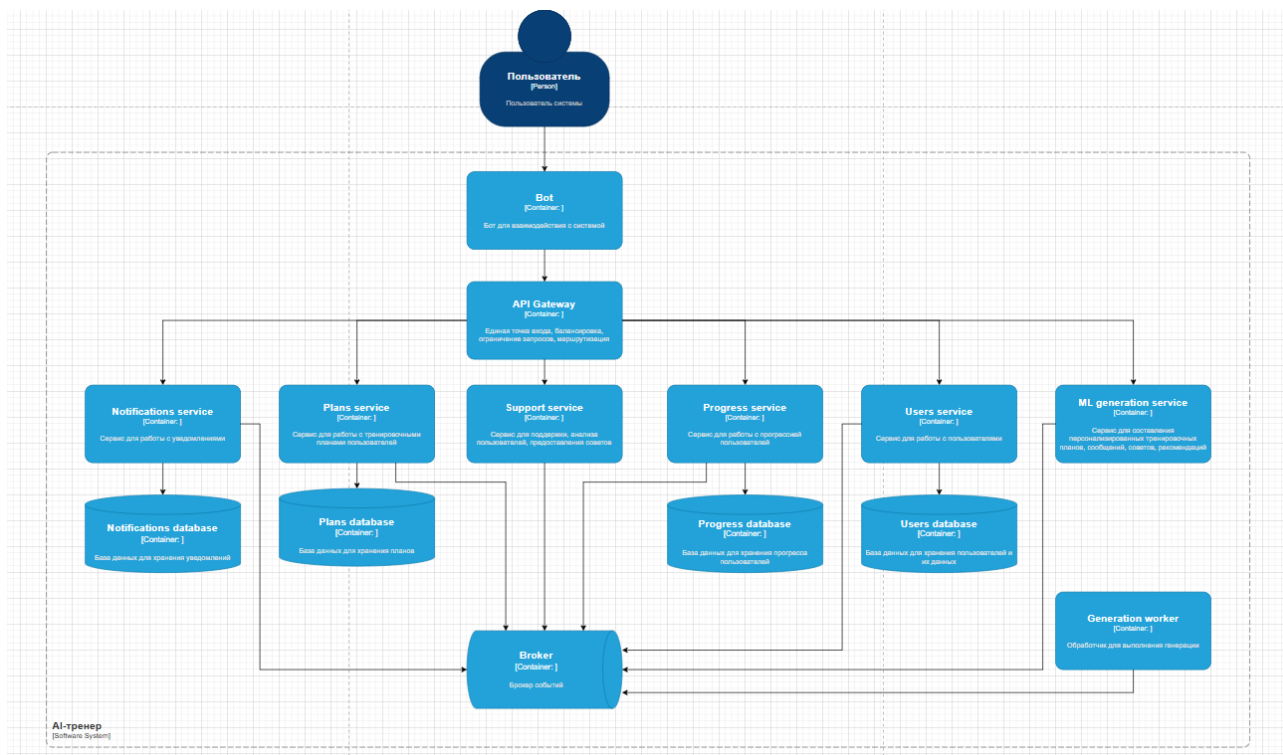


Рисунок 13 – Обновленная диаграмма контейнеров

Обновленная архитектура описана в нотации 4+1. Представления рассмотрены подробнее ниже.

На рисунке 14 представлено логическое представление, классы архитектуры. Система состоит из 5 основных сервисов, определенных по ограниченным контекстам (Users, Notifications, Plans, Progress, Support) и ML сервиса генерации. Выделены абстракции, существующие в системе.

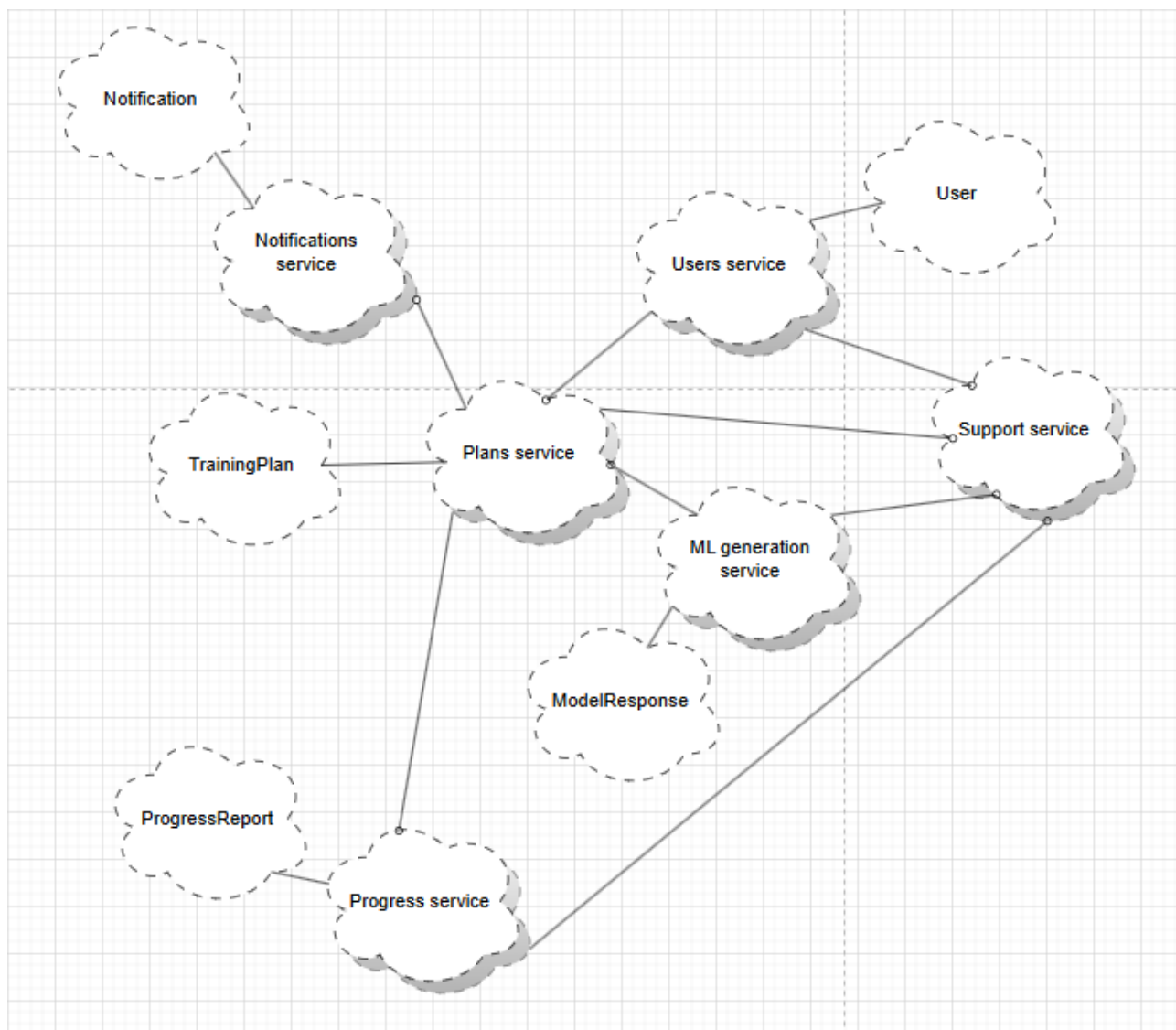


Рисунок 14 – Логическое представление

На рисунке 15 представлено представление разработки. Оно предназначено для отображения внутренних модули сервисного слоя и слоя брокера событий.

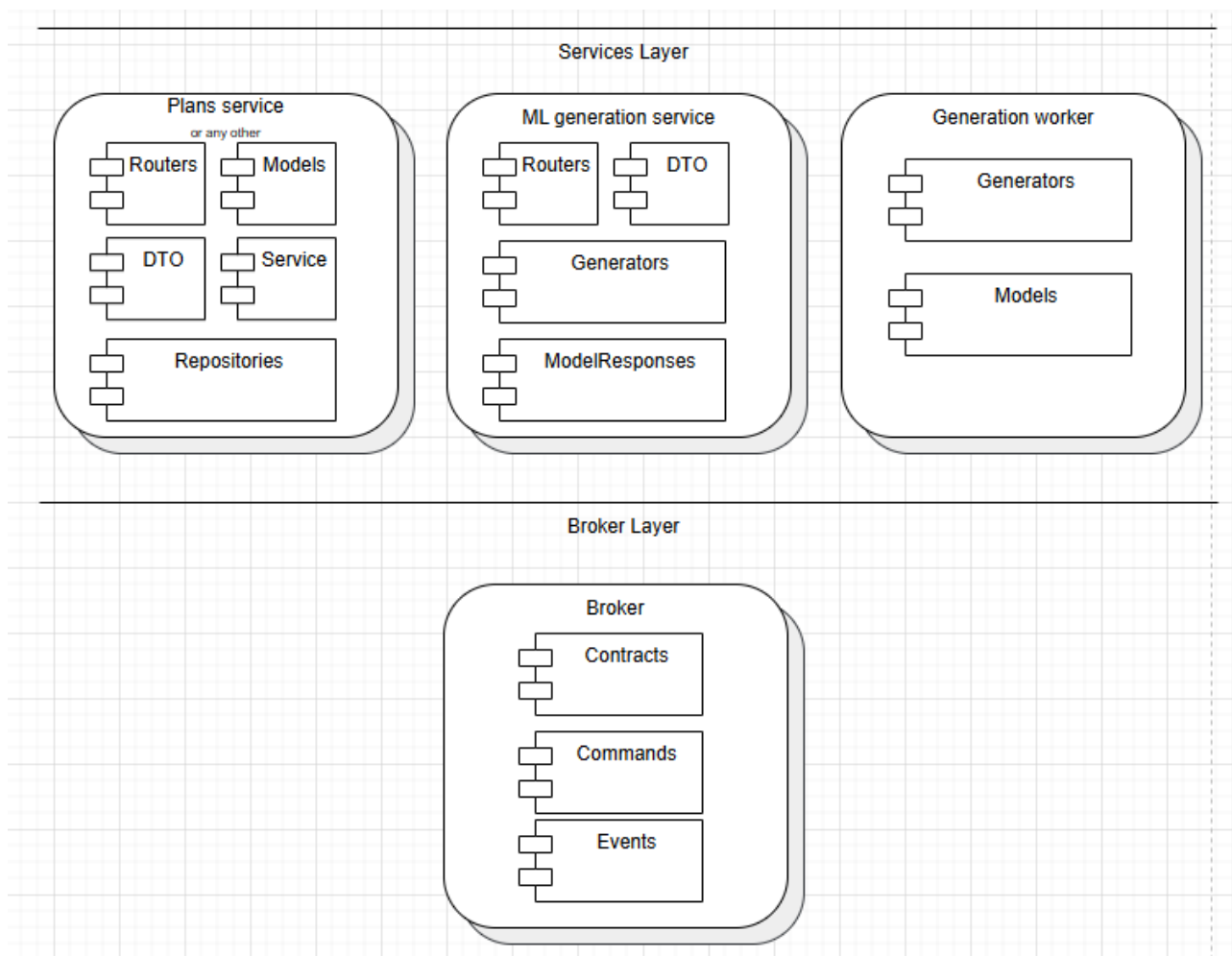


Рисунок 15 – Представление разработки

На рисунке 16 представлено представление процесса, а именно – генерация плана. Как можно увидеть, брокер выступает главным связующим звеном. API Gateway обращается к брокеру с задачей по генерации, затем сервис планов получает это сообщение, проверяет наличие плана, и, если его нет, посылает назад в брокер сообщение о необходимости получения данных пользователя, User service их отправляет, Plan service отправляет сообщение на генерацию. ML сервис генерации собирает информацию и отправляет сообщение для начала генерации для Generation worker. Тот, когда закончит, пошлет готовый план. Остальные сервисы получают уведомления о событиях.

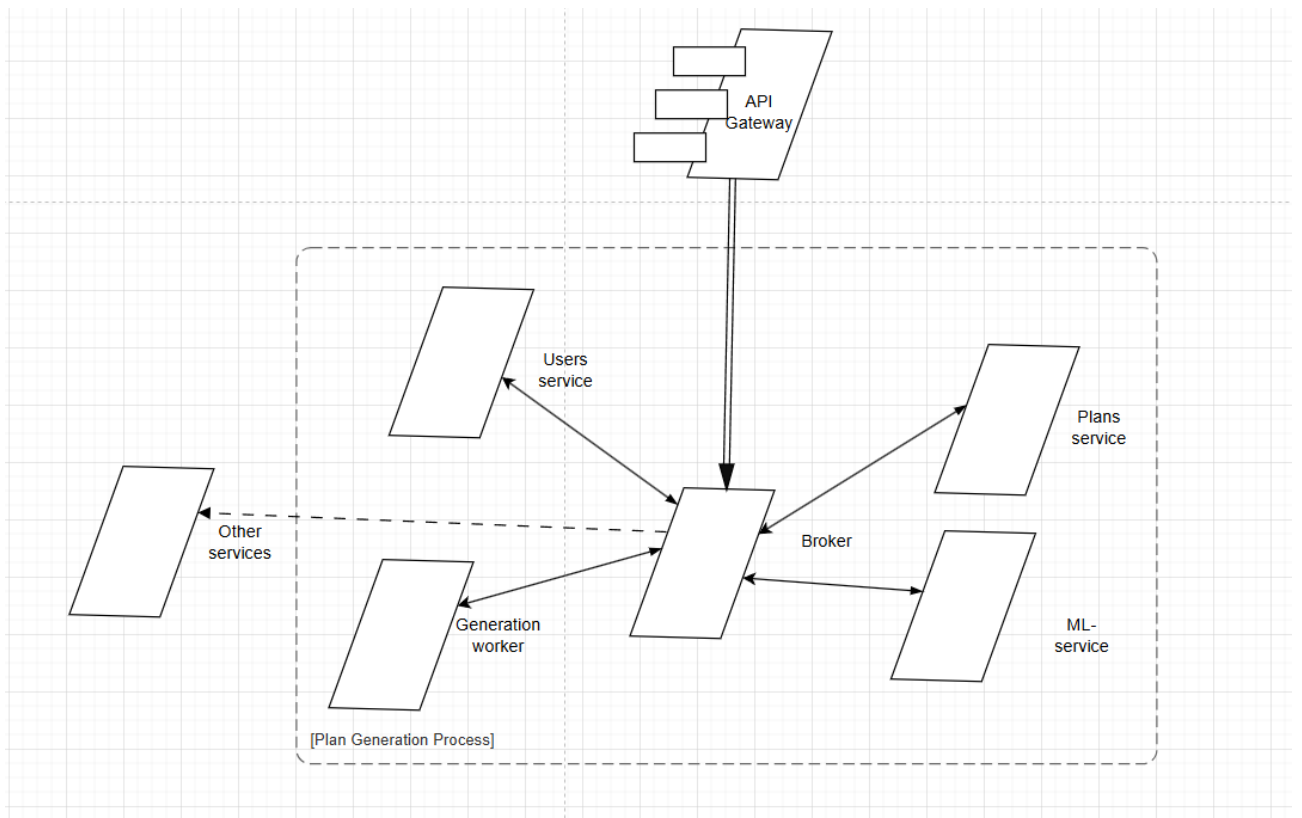


Рисунок 16 – Представление процесса

На рисунке 17 показано физическое представление, которое раскрывает вопрос о сети узлов обработки данных, как устроена система с точки зрения этих узлов обработки, как происходит взаимодействие этих узлов. Данные узлы размещаются на отдельных хостах, сервисы при росте нагрузки можно дублировать.

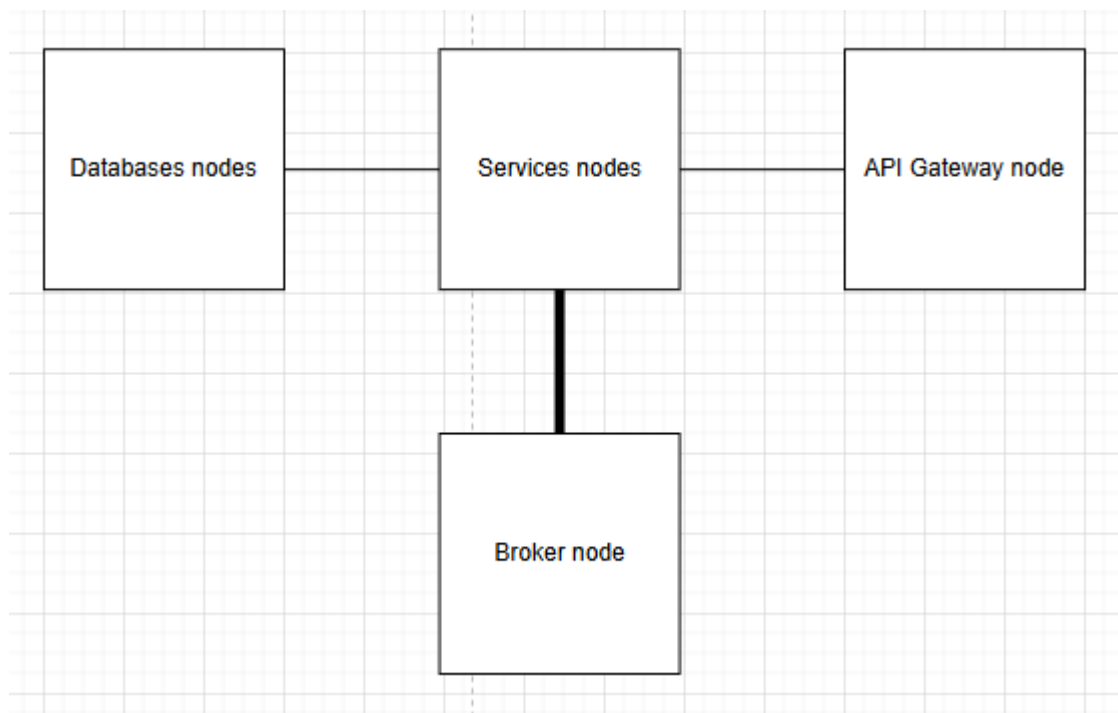


Рисунок 17 – Физическое представление

Сценарий (основной) – пользователь генерирует свой персональный план.

Представлены следующие шаги:

- 1) пользователь отправляет запрос на создание плана;
- 2) API gateway отправляет сообщение в Broker;
- 3) Plan service берет сообщение, проверяет наличие плана, отправляет сообщение на получение данных пользователя;
- 4) User service берет сообщение и отдает данные пользователя;
- 5) Plan service берет информацию, формирует запрос, указывает детали и отправляет запрос на генерацию;
- 6) ML generation service берет данные запроса, настраивает параметры генерации и отправляет запрос на начало генерации для Generation worker;
- 7) Generation worker получает запрос, выполняет генерацию и отправляет результат;
- 8) ML service берет результат и отдает его в установленном формате Plan service;
- 9) Plan service возвращает план с данными клиенту.

3 Выводы

Была выбрана и описана система для разработки архитектуры, к ней разработана диаграмма вариантов использования.

На основе интервью и опросов были описаны требования на основе различных источников на разных уровнях.

Были рассмотрены возможные варианты архитектурных паттернов для системы, обосновано был принят архитектурный паттерн, решение задокументировано в формате ADR.

На основе изученного материала и анализа отобранны и разработаны паттерны проектирования для данной системы с обоснованием их выбора, составлены UML-диаграммы классов.

Были изучены и применены принципы SOLID и принципы организации компонентов, составлены диаграммы кода и компонентов.

Были изучены, рассмотрены и отобранны способы повышения безопасности приложения с анализом причин выбора и компромиссов, связанных с решением.

Выполнен переход к новому архитектурному паттерну на фоне изменения требований. Обновленная архитектура описана в нотации 4+1. Проведена реструктуризация системы.